PHI De-identification Notebook — Complete Guide

Oct 23, 2026 · @Anand

At a glance

This guide explains `final_v3_basic_ocr_vlm_23_oct.ipynb` cell by cell. The notebook takes a scanned healthcare page (CMS-1500, UB-04 or any other form), finds every piece of PHI/PII on it, and writes a realistic **fake** value into the image in the same font, size and position, so the page still looks real but no longer identifies anyone.

The idea in one sentence: Textract tells us *where* every word is, GPT-5.4 tells us *what each value means*, Azure DeID plus our own rules decide *what is PHI*, local code makes *matching fake values*, and pixel-level image code *swaps them in*.

Question	Answer
Input	One scanned page image (PNG/JPG), or a folder of them
Output	`<name>_deid.png` (de-identified page), a side-by-side comparison image, `batch_summary.csv`, and a `<name>_verify.json` record for the leak check
Cloud services	AWS Textract (OCR), Azure OpenAI GPT-5.4 (form understanding), Azure Health Data Services DeID (PHI tagging)
Local work	Matching, rules, fake-value generation, font matching, erasing and redrawing (Python, NumPy, Pillow)
Templates needed?	No. Nothing is tied to a form's coordinates, so new layouts run through the same code

All examples in this guide come from real runs of the notebook's own functions on a mock CMS-1500 page, with the cloud answers simulated. Names, dates and numbers in the examples are invented.

Methods and tech stack

Each step uses the simplest method that does its job reliably; the cloud services are only used where a trained model is needed.

Methods (what technique, for what purpose)

Method	Used for	Where	Why this method
OCR (AWS Textract `DetectDocumentText`)	Read every word with its pixel box, confidence and printed/handwritten flag	Cell 4	Word-level boxes are needed to erase and redraw exactly; basic OCR is the cheapest Textract call (~$0.0015/page)
Coordinate conversion (fraction × image size, floor/ceil)	Turn Textract's 0–1 positions into pixel boxes	Cell 6	Exact, no library needed; rounding outward never cuts a letter
Tilt measurement (`atan2` on the box's top edge)	Know how slanted each word is	Cell 6	Two corners define a direction; used to straighten and to draw rotated
Vision-language model (GPT-5.4)	Understand the form: which value belongs to which label, what is PHI, what is printed form text	Cell 11b	Basic OCR has no context; a VLM reads the form like a person, on any layout
Fuzzy text matching (`difflib.SequenceMatcher`, ≥ 85 %)	Put GPT's answers on the exact OCR words	Cell 11b	GPT gives text, not positions; tolerant to small OCR differences
NER / PHI tagging (Azure Health Data Services DeID, Tag)	Find names, dates, IDs, addresses… in text, with category and confidence	Cells 11, 12	A model trained for healthcare PHI (HIPAA identifiers)

Method	Used for	Where	Why this method
Label rules (`KEY_RULES`, regular expressions)	Value under "BIRTH DATE" → date, under "I.D. NUMBER" → id	Cell 12	Catches bare numbers and codes that a text model cannot classify alone
Patterns (`PATTERNS`, regular expressions)	SSN, phone, email, dates, ZIP anywhere on the page	Cell 12	Fixed formats are always PHI; no model needed
Propagation	A name/ID found once is flagged everywhere it repeats	Cell 12	Azure may tag a name in one box but miss it in another
Union of detectors with protections	A word is PHI if any detector flags it, unless it is protected form text	Cell 12	A miss leaks data; an extra flag only changes a box
Deterministic surrogates (HMAC-SHA256 seed + word lists)	Same-shape fake values; same original → same fake	Cell 13	Consistent across the page and runs; not reversible without the secret key
Date shifting (`datetime`, fixed offset per key)	Move every date by the same number of days	Cell 13	Hides real dates but keeps intervals and ages correct
Otsu thresholding	Separate ink from paper inside each word box	Cell 14	Picks the best cut-off per word, works on any paper colour
Form-line detection	Keep ruled lines when erasing	Cell 14	Lines cross the whole box; letters never do
Template matching with Dice score	Choose the font and size that reproduce the original ink	Cell 14	No training needed; learns from the page itself
Background-sample inpainting	Fill erased ink with real paper pixels from next to the word	Cell 14	Keeps paper texture; no white rectangles

Method	Used for	Where	Why this method
Alpha blending + slight blur	Draw the fake in the original's colour, softened like a scan	Cell 14	Looks printed, not pasted

Tech stack

Technology	Role
Python 3.12 in Jupyter (Windows)	Runtime
AWS Textract via boto3 + company `icp.authenticator` wrapper	OCR; credentials renewed automatically (`@refresh_on_expire`)
Azure OpenAI GPT-5.4 via `openai` (`AzureOpenAI`)	Form understanding; settings from `.env` (`endpoint`, `deployment`, `subscription_key`, `api_version`)
Azure Health Data Services De-identification, REST API 2024-11-15, via `urllib`	PHI tagging
`azure-identity` (`InteractiveBrowserCredential`)	Azure sign-in, token renewed per call
`python-dotenv`, `certifi`, `ssl`	Settings from `.env`; company Netskope certificate for HTTPS
Pillow (PIL)	Load/save images, render fonts, rotate, blur
NumPy	All pixel maths: thresholds, masks, overlap scores, colours
`re`, `bisect`, `difflib`, `math`	Patterns and rules, fast offset lookup, fuzzy matching, angles
`hmac`, `hashlib`, `random`, `datetime`	Deterministic fake values and date shifting
`functools.lru_cache`	Caches rendered text so font matching stays fast
`json`, `csv`, `pathlib`	Batch files, summary CSV, verification records

Technology	Role
TrueType fonts (Courier, Consolas, Arial, Calibri, Times; Liberation/DejaVu on Linux)	Candidates for font matching

Cells 1–5: setup and reading the page

These cells load the tools, read the settings, and turn the image into text with positions.

Cell 1 — Import libraries

```
import boto3, json, os, ssl, sys, tempfile, urllib.request, urllib.error
from collections import Counter
from icp.authenticator.aws.boto3_services import boto3_clients as aws_client
```

Loads the AWS SDK, the company AWS login wrapper and Python's built-in tools for JSON, files, certificates and HTTPS. If anything is missing, the notebook fails here rather than halfway through a batch. Other libraries (Pillow, NumPy, `re`, `openai`…) are imported in the cells that first use them.

Cell 1b — Debug printouts

```
DEBUG = True            # single-image run: print every stage
DEBUG_BATCH = False     # every page of the batch (long output)
DEBUG_MAX_ROWS = 120
```

Defines five printers, one per stage: `debug_ocr`, `debug_vlm`, `debug_tagging`, `debug_surrogates`, `debug_render`. Each later cell calls its printer when `DEBUG` is on. The printouts contain real PHI, so clear the notebook outputs before sharing.

Cell 2 — Azure DeID settings

```
load_dotenv()
ENDPOINT = os.environ["AZURE_DEID_ENDPOINT"]
TENANT_ID = os.environ["AZURE_DEID_TENANT_ID"]
SCOPE = os.environ.get("AZURE_DEID_SCOPE", "https://deid.azure.com/.default")
API_VERSION = os.environ.get("AZURE_DEID_API_VERSION", "2024-11-15")
```

Reads the Azure DeID endpoint and tenant from `.env`, so no secrets live in the notebook. Optional: `DEID_SURROGATE_KEY` in `.env` keeps fake values identical across runs (cell 13).

Cell 3 — Load and check the image

`load_image(path)` checks that the file exists and is PNG, JPG, JPEG, BMP or GIF, and stops with a clear error otherwise. `IMAGE_PATH` is the page used for the single-image walk-through.

Cell 4 — OCR with AWS Textract

```
assert len(image_bytes) < 10 * 1024 * 1024, "Image exceeds 10MB sync limit"

@aws_client.refresh_on_expire
def get_ocr(image_bytes):
    textract = aws_client.get("textract")
    return textract.detect_document_text(Document={'Bytes': image_bytes})
```

- Sends the raw image bytes to Textract's basic OCR (`DetectDocumentText` , synchronous, one page, under 10 MB).
- `@aws_client.refresh_on_expire` renews expired AWS credentials and retries, so long batches do not fail.
- Returns **blocks**: one `PAGE` , one `LINE` per text line, one `WORD` per word. Each word has `Text` , `Confidence` (0–100), `TextType` (PRINTED or HANDWRITING) and `Geometry` (box and 4-corner polygon as fractions of the page).
- The same function later re-reads the de-identified image for the verification record.

Example output:

```
Image size: 842.6 KB
Total blocks returned: 941
Block type counts: {'PAGE': 1, 'LINE': 268, 'WORD': 672}
```

Cell 5 — Print the lines (check only)

Prints every OCR line with confidence and position and writes them to `extracted_text_detailed.txt` , so you can see what Textract read. Nothing later depends on it. The file contains real PHI; keep it protected or remove this step in production.

Cell 6: the word index (the backbone)

`build_word_index()` turns Textract's blocks into two things every later step relies on: `full_text`, the whole page as plain text, and `offset_map`, one entry per word that links its **position in the text** to its **position on the image**. That link is how "Azure says characters 59–64 are a name" becomes "erase pixels (111, 363)–(239, 403)".

What it does, step by step

1. Reads the image size with Pillow (e.g. 2550 × 3300 px for a letter page at 300 DPI).
2. Walks every `LINE` and its child `WORD`s in Textract's reading order.
3. Appends each word to `full_text` with a space, and a newline after each line, recording the word's `start` and `end` character positions.
4. Converts the word's box from fractions to pixels, rounding outward so no letter edge is cut:

```
x0 = floor(Left × width)              y0 = floor(Top × height)
x1 = ceil((Left + Width) × width)     y1 = ceil((Top + Height) × height)
```

5. Measures the tilt from the top edge of the word's polygon, corner P0 to corner P1. Both differences are converted to pixels first, because a page is not square:

```
dx = (P1.X − P0.X) × width;  dy = (P1.Y − P0.Y) × height
angle = degrees(atan2(dy, dx))        # 0 = level; positive = right end lower
```

6. Keeps the confidence and whether Textract read it as handwriting.

Worked example: the surname SMITH on a 2550 × 3300 scan

Textract: `Left 0.0436, Top 0.1101, Width 0.0500, Height 0.0118`; polygon top edge drops 1.2 px over 102 px.

```
{"idx": 9, "start": 59, "end": 64, "text": "SMITH",
 "px": (111, 363, 239, 403),          # floor(0.0436×2550)=111 ...
ceil(0.1219×3300)=403
 "angle": 0.67,                       # degrees(atan2(1.2, 102))
 "confidence": 99.4, "handwriting": False, "line_idx": 1, "word_id":
"8f1c..."}
```

`full_text[59:64]` is exactly `"SMITH"`. `word_by_id` is the same data keyed by `word_id` for quick lookup.

How each field is used later

Field	Used in	For
`start`, `end`	Cell 12	Map Azure's and the patterns' character hits to words
`px`	Cells 11b, 14	Distance between label and value; the rectangle to erase and redraw
`poly`, `angle`	Cell 14	Find ink inside a tilted box; draw the fake tilted (above 0.7°)
`line_idx`, `idx`	Cells 11b, 12, 14	Group words into lines; one font and size per line
`confidence`, `handwriting`	Cells 1b, 14	Shown in the debug printout; handled specially when drawing

Stage 1 debug printout (`debug_ocr` , mock page):

```
line  min_conf  text
low_conf_words
0     99        HEALTH INSURANCE CLAIM FORM
1     99        APPROVED BY NATIONAL UNIFORM CLAIM COMMITTEE (NUCC) 02/12
2     99        1. MEDICARE MEDICAID TRICARE CHAMPVA
6     99        CAROLINE MARTIN, KATHLEEN
7     99        MM 05 DD 08 YY 1985
10    99        RIDGE, AL 026037671
12    64        38784244
LOW:38784244
13    99        NUCC Instruction Manual available at: www.nucc.org
words: 68 | lines: 14 | low-confidence (<70) words: 1 | handwritten: 0
```

Cells 8–11: label: value text, network, sign-in, Azure DeID call

Section 7 (manual layout pairing) was removed in this version: label → value pairs now come only from GPT-5.4 (cell 11b).

Cell 8 — `build_context_view()` (function only)

Writes one `label: value` line per pair and records, for every value word, where it sits in that text and its `word_id` . It is called from cell 11b with GPT's pairs.

```
def build_context_view(kv_pairs, table_cells, word_by_id):
    for each pair:   ctx_text += label + ": "
                     for each value word: record {start, end, word_id};
ctx_text += word + " "
                     ctx_text += "\n"
    return ctx_text, ctx_map
```

Why: Azure DeID is a text model and judges a value by the words around it. On a CMS-1500 the value is printed far from its label, so in plain reading order `123456789` stands alone. In the context text Azure sees `1a. INSURED'S I.D. NUMBER: 123456789`. Only value words go into `ctx_map`, so a tag on a label word never changes the image.

Cell 9 — Certificate bundle (office network)

The company Netskope proxy re-signs HTTPS traffic with its own certificate, which Python does not trust by default (`CERTIFICATE_VERIFY_FAILED`). `export_netskope_ca()` reads the Windows certificate store (`ssl.enum_certificates`), keeps Netskope's root, appends it to `certifi` 's list in a temp file, and points `REQUESTS_CA_BUNDLE` , `SSL_CERT_FILE` and `AWS_CA_BUNDLE` at it. HTTPS checking stays on; the proxy is simply trusted.

Cell 10 — Azure sign-in

`InteractiveBrowserCredential(tenant_id=TENANT_ID)` opens a browser login once (the account needs the DeID Data Owner role). Later, `credential.get_token(SCOPE)` returns a cached token and renews it silently before it expires.

Cell 11 — The Azure DeID call

```
def call_azure_deidentification(text, operation="Tag"):
    token = credential.get_token(SCOPE)
    body = json.dumps({"inputText": text, "operation": operation}).encode()
    POST f"{ENDPOINT}/deid?api-version={API_VERSION}"  (Bearer token, JSON,
60 s timeout)
```

- **Tag** returns where PHI is (character offsets), its category and a confidence. That is all we need; the fake values are made locally in cell 13.
- **`azure_tag(text)`** splits long text, calls Tag per piece, converts Azure's positions to Python positions (`codePoint`), checks each entity's text really sits at that position (and re-finds it nearby if not), and returns `[{start, end, category, confidence,`

`text}]`.

Chunking (`_split_chunks`): Azure accepts at most 50 KB per request, so text is sent in pieces of up to 20,000 characters, always cut at the last line break before the limit, so no name or date is split. Each piece remembers where it started (`base`), and Azure's positions are shifted back by that amount. Example with a limit of 60 characters:

Chunk	Starts at	Text
1	0	`2. PATIENT'S NAME: SMITH JOHN A`
2	33	`3. PATIENT'S BIRTH DATE: 03 15 1980`
3	70	`5. PATIENT'S ADDRESS: 123 MAIN ST` + `CITY: HARRISBURG`

Azure finds `03 15 1980` at position 25 in chunk 2 → 33 + 25 = 58 in the full text. A CMS-1500 (~5,000 characters) or UB-04 (~7,500) fits in one chunk, so normally there are two calls per page: one for `full_text`, one for `ctx_text`. Azure bills by MB of text, not per call, so chunking adds no cost.

Example Azure answer (shortened):

```
{"taggerResult": {"entities": [
  {"category": "Patient", "offset": {"codePoint": 59}, "length":
{"codePoint": 12}, "text": "SMITH JOHN A", "confidenceScore": 0.96},
  {"category": "Date", "offset": {"codePoint": 98}, "length": {"codePoint":
10}, "text": "03 15 1980", "confidenceScore": 0.99}]}}
```

Cell 11b: GPT-5.4 gives the context

Basic OCR knows where words are but not what they mean. GPT-5.4 looks at the page image and returns every filled-in field (label, value, category, PHI or not), PHI in free text, and the form's printed text. Each answer is matched back to the OCR words, so it lands on exact pixel boxes.

Settings (from `.env`: `endpoint`, `deployment`, `subscription_key`, `api_version`; optional `model_name`)

```
USE_VLM = True; VLM_MAX_WIDTH = 2000; VLM_MAX_TOKENS = 16000;
VLM_MATCH_THRESHOLD = 0.85; VLM_CACHE_DIR = None
```

The prompt (summary). The full text is `VLM_PROMPT` in the cell. It asks for JSON only:

```
{"fields": [{"label": "...", "value": "...", "category": "...", "is_phi":
true}],
 "free_text_phi": [{"value": "...", "category": "..."}],
 "printed_text": ["..."]}
```

- `label`: the printed field name exactly as printed, with its box number (`10 BIRTHDATE`, `45 SERV. DATE`).
- `value`: copied **exactly** as printed, never reformatted, so it can be matched to the OCR text.
- One entry per table cell (UB-04 service lines); addresses split into street, city, state, ZIP.
- `category` from a fixed list (name, date, ssn, health_plan_id, street, city, state, zip…), mapped to the notebook's kinds by `VLM_KIND`.
- `is_phi`: true for identifiers of people and providers; false for diagnosis/procedure codes, charges, checkboxes, sex, relationship.
- `printed_text`: every pre-printed line of the blank form (titles, captions, checkbox labels, footers).

Functions

Function	What it does
`vlm_extract(image_path)`	Resizes the page to ⩽ 2000 px wide, sends it with the prompt (detail high), parses the JSON; stops with a clear error if the answer was cut off
`match_text(text, offset_map)`	Finds where a text is printed: windows of n−1 to n+1 consecutive OCR words on one line, compared after removing case and punctuation; ⩾ 85 % similar (values of ⩽ 3 characters must match exactly). Date captions (`MM`, `DD`, `YY`) are skipped while matching
`vlm_pairs_and_hits(vlm, offset_map)`	For each field: find its label, find its value (the occurrence closest to the label), build a pair; PHI values become hits for cell 12. Free-text PHI becomes hits too. Printed text is matched and protected

Function	What it does
`boilerplate_word_ids()`	Protects known CMS-1500 / UB-04 printed text (`BOILERPLATE_PHRASES` : HEALTH INSURANCE CLAIM FORM, NUCC, MEDICARE, MEDICAID, SSN, EIN, PICA, MM DD YY, the footer …) even if GPT does not list it
`vlm_build_context()`	Builds `kv_pairs` , `label_word_ids` , `ctx_text` , `ctx_map` , `vlm_hits` and the protected set from GPT's answer only

If the GPT call fails, the page continues without label context: Azure on the full text, the patterns and propagation still run, and the batch summary records `context_source = none` .

Example: what GPT returned and how it matched (Stage 2 debug printout, mock page)

label	value	category	is_phi	matched to OCR
2. PATIENT'S NAME	CAROLINE MARTIN, KATHLEEN	name	True	yes: CAROLINE MARTIN, KATHLEEN
3. PATIENT'S BIRTH DATE	05 08 1985	date	True	yes: 05 08 1985 (captions MM/DD/YY skipped)
32. SERVICE FACILITY LOCATION INFORMATION	RIDGE	city	True	yes: RIDGE,
32. SERVICE FACILITY LOCATION INFORMATION	026037671	zip	True	yes: 026037671
26. PATIENT'S ACCOUNT NO.	38784244	account_number	True	yes: 38784244

```
printed form text lines from GPT: 5 (matched: 5)
protected (never changed) words: 52 -> (Medicaid#) (Medicare#) (NUCC) 02/12
APPROVED CHAMPVA CLAIM COMMITTEE DD
   FORM HEALTH INSURANCE MEDICAID MEDICARE MM NATIONAL RESERVED TRICARE
UNIFORM USE YY www.nucc.org ...
```

The text then sent to Azure (`ctx_text`):

```
2. PATIENT'S NAME: CAROLINE MARTIN, KATHLEEN
3. PATIENT'S BIRTH DATE: 05 08 1985
32. SERVICE FACILITY LOCATION INFORMATION: RIDGE,
32. SERVICE FACILITY LOCATION INFORMATION: 026037671
26. PATIENT'S ACCOUNT NO.: 38784244
```

Cell 12: deciding what is PHI

`detect_phi()` marks a word as PHI if **any** detector flags it, unless the word is protected form text. A missed value leaks real data; an extra flag only means a box gets a fake it did not need, so the design favours catching too much over too little.

The detectors

#	Detector	How it decides	Example
1	Azure Tag on `full_text`	Azure's model on the whole page in reading order	a name in a free-text note
2	Azure Tag on `ctx_text`	Same model, each value next to its label (from GPT)	`1a. INSURED'S I.D. NUMBER: 123456789`
3	Label rules (`KEY_RULES`)	Value under a label containing e.g. birth / DOB → date; ID, member, policy, account, MRN, NPI → id; address → street; city / state / zip / country; phone; name / signature → name	`38784244` under "PATIENT'S ACCOUNT NO."
3b	GPT-5.4 hits (source `vlm`)	Values GPT marked as PHI, matched to OCR words	`031580` under UB-04 `10 BIRTHDATE`

#	Detector	How it decides	Example
4	Patterns (`PATTERNS`)	SSN `123-45-6789`, phone, email, dates (4 formats), ZIP after a state code (5, 5+4 or 9 digits)	`AL 026037671`
5	Propagation	A name, ID, phone, email or city found once is flagged wherever it repeats (letters 3+, digits 4+)	the patient name again in the signature box

Azure's categories are mapped to kinds by `CATEGORY_KINDS` (Patient/Doctor → name, CountryOrRegion → country, anything unlisted such as MedicalRecord or HealthPlan → id). Ages count only above 89, as in HIPAA Safe Harbor. A value must also look plausible for its label (`_token_ok`): a date needs a digit, an ID a digit and 3+ characters, a ZIP 5+ digits.

How one detection is recorded: `flag()`

```
def flag(wid, kind, source, conf=1.0, span=None):
    if word in FORM_TOKENS:            -> stop  ("form caption token": MM,
DD, YY, YES, NO, NPI, SSN, EIN, PICA ...)
    if word in hard_labels:            -> stop  ("printed form text": GPT
labels/printed text + boilerplate)
    if word in label_word_ids and not strong:  -> stop  ("label word")
    flags[wid] = {kind, sources, conf, mask}  # mask = which characters are
PHI
```

- The **mask** lets `DOB:03/15/1980` keep `DOB:` and change only the date.
- When detectors disagree on the kind, the more reliable one wins: pattern > Azure = GPT > rule > propagation (`PRIORITY`).
- Only a pattern match for an SSN, phone or email may override protected text.
- Every stopped attempt is recorded for the Stage 3c printout.

After all detectors run, a credential right after a name (`SMITH MD`, `JONES PA`) is un-flagged as a state, and neighbouring flagged words of the same kind on one line are grouped into **entities** (`CAROLINE MARTIN, KATHLEEN` = one name).

Real run on the mock page. Azure was simulated to tag printed form text the way it did on the real page you showed (title, NUCC line, MEDICARE…).

Stage 3b — words flagged as PHI:

line	word	kind	found by
6	CAROLINE / MARTIN, / KATHLEEN	name	azure, azure_ctx, rule, vlm
7	05 / 08 / 1985	date	azure, rule, vlm
10	RIDGE,	city	rule, vlm
10	026037671	zip	pattern, rule, vlm
12	38784244	id	rule, vlm

Stage 3c — protected words a detector tried to change (all stopped):

word	Azure said	conf	why stopped
NATIONAL UNIFORM CLAIM COMMITTEE	Organization	0.91	printed form text
NUCC	Organization	0.86	printed form text
02/12	Date	0.90	printed form text
MEDICARE / MEDICAID	HealthPlan	0.88 / 0.87	printed form text
INSURANCE	Organization	0.84	printed form text
www.nucc.org	Url	0.95	printed form text
MM / DD / YY	Date	0.93	form caption token

Before these protections, exactly these words were being replaced on the real page ("HEALTH MASAMEWIM AWAPUFORM", "FRANKLIN MADISON BIRCH CLEVELAND (WOBU) 07/11", "AKAVEWUB LENEGAWA" …).

PII coverage. Names, dates, ages over 89, addresses, phone, fax, email, SSN, member/policy/account/MRN/claim/license/NPI/tax numbers, organisations, professions, URLs. Card and bank numbers, passport numbers and IP addresses rely on Azure and GPT only. Sex, race, marital status and relationship are intentionally not replaced.

Cell 13: how the fake values (surrogates) are made

Every PHI entity gets a realistic fake with the **same shape** as the original: same length, same upper/lower case, digits stay digits, punctuation stays in place. That is what lets the fake fit the original box on the image.

Two rules every fake follows

1. **Same original → same fake, everywhere.** `_rng()` seeds a random generator with an HMAC-SHA256 of the original value and a secret key:

```
digest = hmac.new(SURROGATE_KEY, "name|SMITH".encode(),
hashlib.sha256).digest()
rng = random.Random(int.from_bytes(digest[:8], "big"))     # same input ->
same choices, every time
```

The key comes from `DEID_SURROGATE_KEY` in `.env`. With the same key, SMITH becomes the same fake on every page, in every run. Without the key, nobody can trace a fake back to its original. With no key set, a random one is used per session.

2. **Never the original.** Every generator excludes the original from its choices.

How each kind is faked (`surrogate_core()`, `phrase_surrogate()`)

Kind	Method	Example from a real run
Name	Same-length real names from `_NAMES` → the 5 closest in printed width → seeded pick → original's case	`SMITH` → `DIANA`, `Smith` → `Diana`, `CAROLINE MARTIN, KATHLEEN` → `BENJAMIN BISHOP, ROBINSON`
Street / city / organisation	Same method with place words (`_PLACES`); words like ST, AVE, APT, SUITE, HOSPITAL kept	`123 MAIN ST` → `680 KING ST`, `HARRISBURG` → `HUNTINGTON`, `RIDGE,` → `MAPLE,`
State / country / profession	A real one from `PHRASE_POOLS` with the same number of words; a 2-letter state code stays a code	`PA` → `AR`, `New York` → `South Dakota`, `India` → `Ghana`, `Teacher` → `Janitor`
Date	Moved by **one fixed number of days** for the whole run (`DATE_SHIFT_DAYS`, 1–364 days back, fixed by the key); same format, 17 formats recognised	shift −4: `03 15 1980` → `03 11 1980`, `04/02/2024` → `03/29/2024`, `March 5, 2021` → `March 1, 2021`, `031524` → `031124`

Kind	Method	Example from a real run
Date in 3 boxes	`MM | DD | YY` side by side are shifted as **one** date, then split back (`_merge_split_dates`)	`05 | 08 | 1985` → `05 | 04 | 1985`
ID, phone, ZIP, SSN	Each digit run → seeded random digits of the same length; letters → letters; punctuation kept	`123456789` → `080808334` , `(717) 555-0143` → `(406) 068-9871` , `XJ445566` → `WL564291` , `026037671` → `522952911`
Email	Letters and digits changed; `@` , dots and `.com` kept	`john.smith@mail.com` → `mule.ripog@pule.com`
No identity	Kept as is: JR, SR, MD, DR, MRS, ST, AVE, PO BOX, a year on its own, month names	`ST` → `ST`

Example: how SMITH becomes DIANA

1. SMITH has 5 letters → the 58 five-letter names in the list (minus SMITH).
2. Printed width of SMITH ≈ 3111 units → keep the 5 closest: LEWIS 3112, KELLY 3113, BRIAN 3056, DAVID 3056, DIANA 3056. Far ones like BROWN 3833 are dropped.
3. The seeded generator for "name|SMITH" picks DIANA, and picks it again every time.
4. Case is copied: SMITH → DIANA, Smith → Diana.

Why dates are shifted, not randomised. All dates move together, so the gaps between them stay true (admitted 03/01, discharged 03/05 is still 4 days) and the patient's age at the visit is unchanged, while the real dates are hidden. Keep the key secret: with it, the shift could be undone.

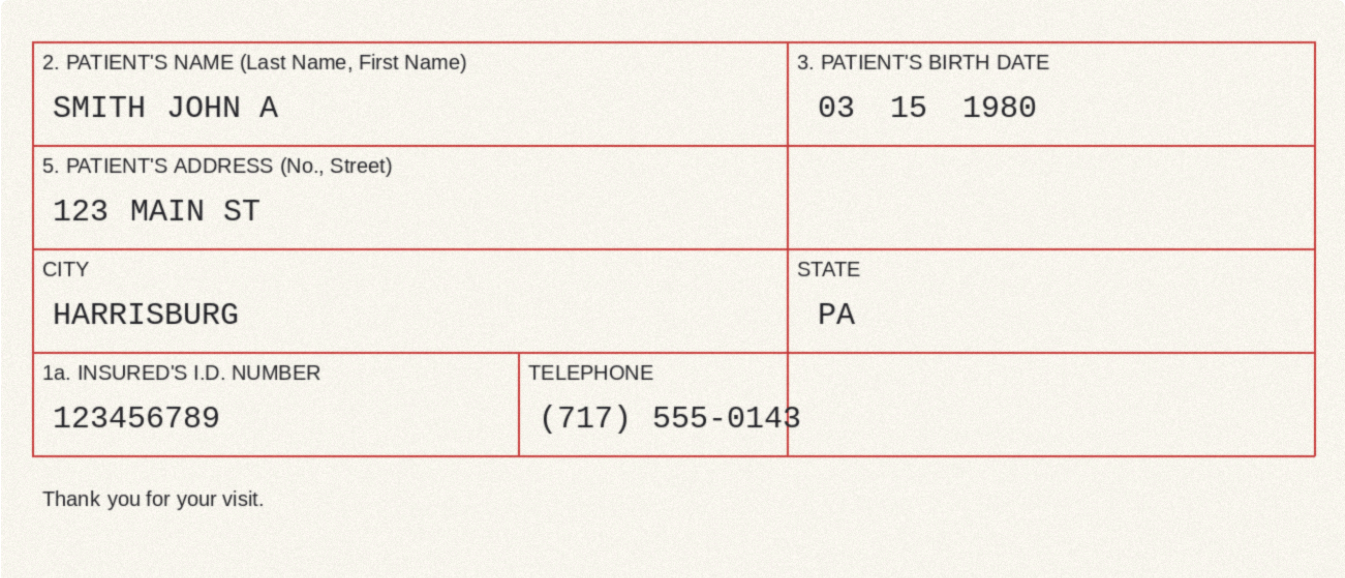
`build_surrogates()` splits each word into kept prefix, PHI part and kept suffix (from the mask), merges 3-box dates, then writes `new_text` on every word and `surrogate` on every entity.

Stage 4 debug printout (mock page):

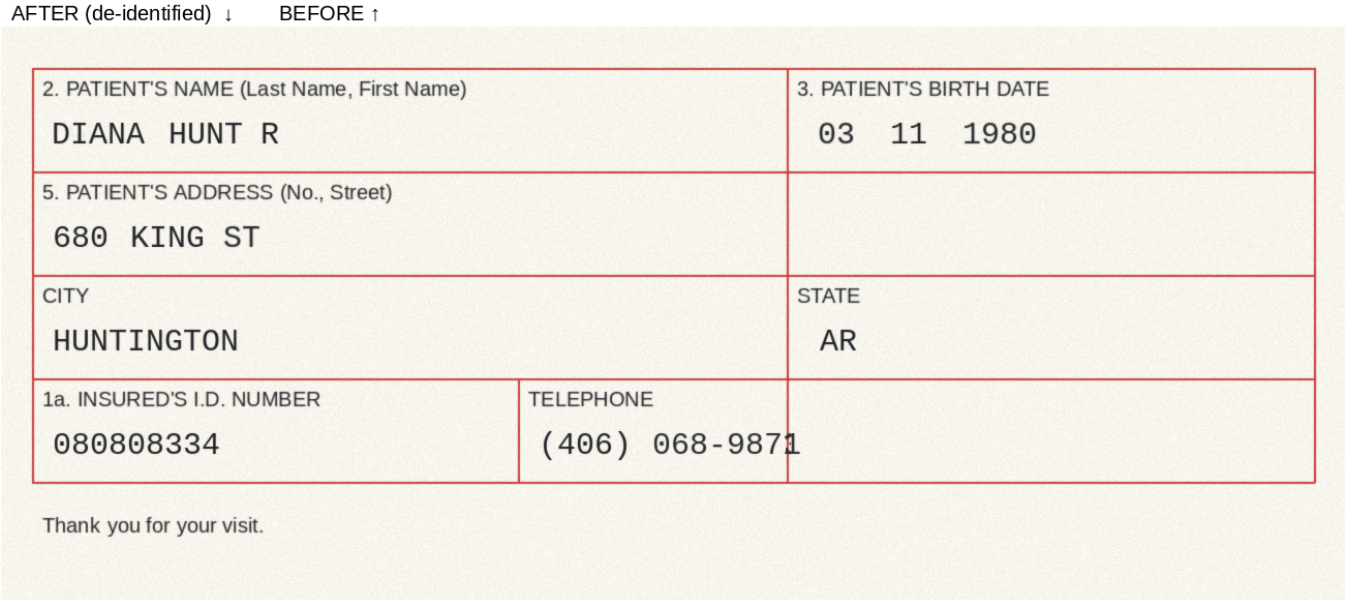
```
kind  original                    fake                         found_by
name  CAROLINE MARTIN, KATHLEEN   BENJAMIN BISHOP, ROBINSON
azure,azure_ctx,rule,vlm
date  05 | 08 | 1985              05 | 04 | 1985              azure,rule,vlm
city  RIDGE,                      MAPLE,                       rule,vlm
zip   026037671                   522952911                    pattern,rule,vlm
id    38784244                    48315739                     rule,vlm
```

Cell 14: putting the fakes on the image

`render_page()` follows one principle: learn the font and size from the original ink, then swap the text. Form lines, paper texture and spacing stay as they were.



AFTER (de-identified) ↓ BEFORE ↑



Real run on a mock CMS-1500 (values in a Courier-style font at size 30, red form lines, scan blur). Result: `Main font: LiberationMono-Regular | Words: {'replaced': 11, 'unchanged': 3}`. The 3 unchanged are `03`, `1980` and `ST`, whose fakes equal the original. A red form line ran through the phone number's last digit; it survived the erase.

Step by step

1. **Find the ink** (`analyze_word`): crop the word's pixel box from the original image (+2 px), keep only pixels inside its tilted polygon, split ink from paper with **Otsu's threshold** (the grey level that best separates dark from light, computed per word), and remove **form lines** (`_find_form_lines`: a run of dark pixels that enters at one edge and leaves at the opposite edge is a ruled line, never a letter). Result for SMITH: OCR box (49, 89, 140, 117) → ink box (51, 93, 138, 113).
2. **Choose the font** (`choose_styles` → `fit_size` → `score_fit`): this is **template matching**. The original word is drawn in every candidate font at the size that makes it exactly as wide as the real ink, then overlaid on the ink and scored:

```
Dice  = 2 × (ink in common) / (ink in drawing + ink in original)      # after
a 0.6 px blur
shape = height penalty × width penalty   (1 px free, then 2× each extra px /
size)
score = Dice × shape                          # 0 = nothing alike, 1 = identical
```

Font (for SMITH)	Size	Score
LiberationMono-Regular (the true font)	**30** (the true size)	**0.916**
LiberationMono-Bold	30	0.838
DejaVuSansMono	30	0.743
LiberationSans-Regular	29	0.553

The decision is made per **line**: average score of the line's words, plus a small bonus (≤ 0.15, `FONT_PEER_WEIGHT`) for the font that wins most often among same-size words elsewhere on the page. Lines of mostly digits borrow the font of the nearest lettered line, because digits look alike in many fonts. Size = the median fitted size on the line.

3. **Colour** (`ink_colour`): the fake gets the same total darkness as the original; scanned ink is dark grey, not pure black.

4. **Position** (`word_geometry`): start at the original ink's left edge, on its measured baseline.
5. **Layout**: consecutive replaced words keep their original gaps; a longer fake is squeezed sideways (never a smaller font) so it stops before the next printed word.
6. **Erase, then draw** (`erase_word` , `draw_word`): all words are erased first, then all drawn. Erasing removes only ink pixels (never form lines) and fills them with real paper pixels sampled next to the word. Drawing renders the fake, rotates it if the word is tilted over 0.7°, blurs it 0.45 px like a scan, and blends it in the measured colour.

The PNG is saved from new pixels only (no metadata from the source file); a grey scan stays grey.

Settings

Setting	Value	Meaning
`MIN_OCR_CONF`	70	Below this, OCR text is considered unreliable
`KEEP_LOW_CONFIDENCE`	False	Low-confidence PHI is replaced too (True would leave it visible)
`ROTATE_MIN_DEG`	0.7	Above this, the fake is drawn tilted
`MAX_SKEW_DEG`	3.0	Strongly tilted words are left out of font learning
`CALIB_WORDS`	60	Extra page words used to learn the fonts
`BLUR_RADIUS`	0.45	Scan-like softness
`EXTRA_FONT_PATHS`	[]	Add `.ttf` files if your forms use a font not in the list

Stage 5 debug printout: one row per PHI word with the original, the drawn fake and the action (`replaced` , `replaced (condensed)` , `unchanged`).

Batch run and outputs

The last cells run the whole pipeline for every image in a folder: `BATCH_INPUT_DIR = Input_docs\data` , searched recursively.

`process_image_pipeline(image_path)` for one page, in order:

```
response = get_ocr(image_bytes)                                    # Textract
basic OCR
full_text, offset_map = build_word_index(blocks, w, h)             # word
index
kv_pairs, label_word_ids, ctx_text, ctx_map, vlm_hits, vlm_label_ids,
vlm_stats = vlm_build_context(...)   # GPT-5.4
vlm_label_ids = protected_form_ids(offset_map, vlm_label_ids)     # + CMS-
1500 / UB-04 boilerplate
flags, entities, azure_entities = detect_phi(..., vlm_hits=vlm_hits,
vlm_label_ids=vlm_label_ids)   # Azure + rules + patterns
build_surrogates(flags, entities)                                  # fake
values
deid_image, actions, doc_font = render_page(image_path, offset_map, flags)
# redraw
# save PNG, comparison image, verification record; one row in
batch_summary.csv
```

- `discover_images()` skips earlier outputs (`output_images` folders, `_deid` / `_comparison` files).
- One failing page is logged to `batch_errors.csv` ; the loop continues.
- If the GPT call fails, that page continues without label context and is marked in the summary.
- `DEBUG_BATCH = True` prints all five debug stages for every page.

Files per page

File	Folder	Contents
`<name>_deid.png`	`output_images` next to the input	The de-identified page
`<name>_comparison.png`	`Input_docs\compared_data`	Original and de-identified side by side
row in `batch_summary.csv`	`Input_docs\compared_data`	counts and the columns below
`<name>_verify.json`	`Input_docs\verification_records`	Original words, what was flagged, the fake values, what happened on the image, and a re-OCR of the output

Useful `batch_summary.csv` columns

Column	Meaning
`phi_words`, `phi_entities`	How much PHI was found
`replaced_words`, `unchanged_words`, `kept_low_confidence_words`	What happened on the image
`context_source`	`vlm` if GPT gave the context, `none` if the call failed
`vlm_fields`, `vlm_fields_matched`	Fields GPT returned / found among the OCR words
`vlm_phi_words`, `vlm_only_phi_words`	PHI words GPT helped find / only GPT found (the value GPT adds)
`protected_words_blocked`	Attempts to change printed form text that were stopped
`vlm_error`	Why GPT failed on that page, if it did
`font_used`, `verification_record`	Main font chosen; path of the verification record

The verification record is what `phi_leak_verification_14_oct.ipynb` uses to find missed PHI: for every PHI value on the original, it checks whether the same text is still readable at the same position on the output. It holds each real value next to its fake, so it is a re-identification key: keep `verification_records` as protected as the original images and never share it. `SAVE_VERIFICATION_RECORD = False` turns it off (one Textract call less per page).

Settings, limitations and how to check it works

Settings you will actually touch

Setting	Cell	Default	What it changes
`DEBUG` / `DEBUG_BATCH`	1b / batch	True / False	Print the stage outputs (OCR, GPT, tagging, surrogates, rendering) for the single page / every batch page
`USE_VLM`	11b	True	Turn the GPT-5.4 step off (no label context, no GPT hits)
`VLM_MAX_SIDE`, `VLM_CACHE_DIR`	11b	2000, None	Image size sent to GPT; a folder to reuse GPT answers when re-running the same pages
`BOILERPLATE_PHRASES`	11b	CMS-1500 / UB-04 titles	Printed text that is never changed. Add the titles of new form types here
`FORM_TOKENS`	12	MM, DD, YY, M, F, YES, NO, NPI…	Single caption words that are never changed
`SKIP_KINDS`	12	empty	Azure categories to ignore (empty = state, country and profession are treated as PHI)
`MIN_AZURE_CONF`	12	0.0	Ignore Azure entities below this confidence
`LABEL_OVERRIDE_CONF`	12	0.80	Azure must be this sure to change a word that looks like a label
`PROPAGATE_KINDS`	12	name, id, phone…	Same value found elsewhere on the page is also replaced
`DEID_SURROGATE_KEY`	13	from `.env`	Secret that makes fake values repeatable. Same key = same fakes; keep it secret

Setting	Cell	Default	What it changes
`KEEP_LOW_CONFIDENCE`	14	False	True would leave words the OCR was unsure about unchanged (not recommended)
`EXTRA_FONT_PATHS`	14	empty	Add your own .ttf fonts to the font-matching candidates
`SAVE_VERIFICATION_RECORD`	batch	True	Write the `*_verify.json` file used for leak checks

Known limitations

- If the GPT call fails (timeout, content filter), the page still runs on Azure + rules + patterns but without label context. `context_source = none` and `vlm_error` show it; re-run those pages.
- QR codes and barcodes are not touched (removed by your choice). They can encode the patient ID.
- Handwriting is replaced with a printed font, so handwritten fields look different from the original.
- Photos of screens or skewed phone photos reduce OCR quality; a word the OCR misses cannot be replaced.
- Card numbers, bank accounts, passport numbers and IP addresses have no regex pattern; they depend on Azure and GPT finding them.
- One page per call; multi-page PDFs need to be split into images first.
- The batch runs pages one by one. For large volumes, run several pages in parallel (threads) within the Azure and Textract rate limits.
- Azure sign-in is interactive (browser). For unattended runs, switch to a service principal.

How to check that it works

1. Run one page with `DEBUG = True` and read the five stages: is every value in the OCR output, did GPT return it, was it tagged, did it get a fake value, was it drawn?
2. Look at the comparison image: printed titles, labels and MM / DD / YY captions must be unchanged.

3. In `batch_summary.csv`, check `context_source`, `vlm_error`, `unchanged_words` and `protected_words_blocked` for every page.
4. Run `phi_leak_verification_14_oct.ipynb` on the verification records. Any `LEAK - NOT DETECTED` or `DETECTED NOT REPLACED` is a page to look at.
5. Because GPT is now part of detection, also check a few pages against a list of PHI written by hand. Using GPT alone to check GPT's own work would miss what it misses.